

Introduction to Maven

What is Maven?

Maven is a powerful build automation and project management tool primarily used for Java projects. It simplifies the build process, dependency management, and project documentation by using a standard configuration.

Key Features of Maven

- **Dependency Management:** Automatically downloads and manages libraries required for the project.
- **Build Automation:** Automates compiling, packaging, testing, and deploying code.
- **Standardized Project Structure:** Uses a standard directory layout.
- **Extensible via Plugins:** Supports numerous plugins to add functionality.
- **Integration with IDEs:** Easily integrates with popular IDEs like IntelliJ IDEA, Eclipse, NetBeans.
- **Repositories:** Uses repositories (local, central, remote) to store and retrieve dependencies.

Maven Architecture

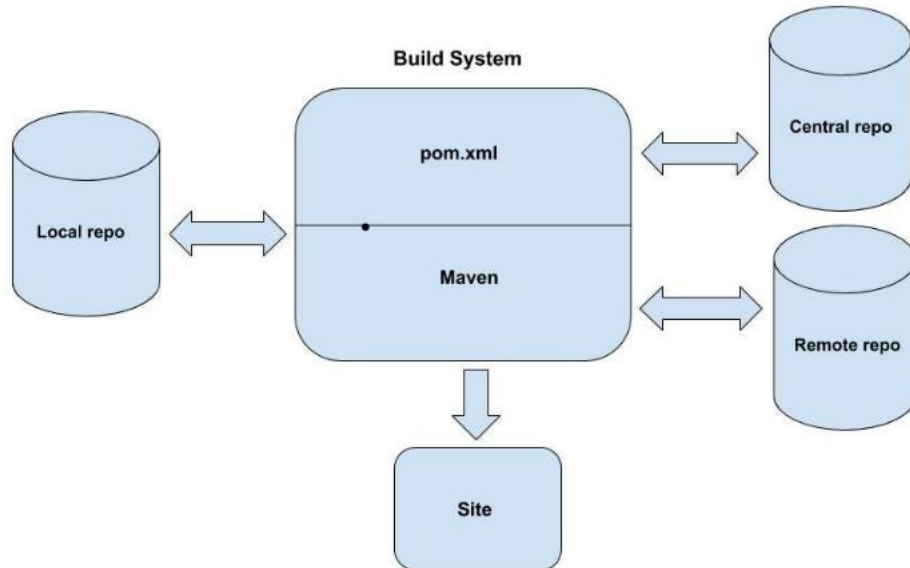


Image Placeholder: Maven Architecture

+-----+
| Developer |

+-----+
|
v

+-----+
| POM (Project Object Model) |

+-----+
|
v

+-----+
| Build Lifecycle |

+-----+
|
v

+-----+
| Plugins & Goals |

+-----+
|
v

+-----+
| Dependencies/Repos |

+-----+

Maven Project Structure

project

```
|
|
| └─ src
|   |
|   └─ main
|       |
|       └─ java
|           |
|           └─ resources
|
| └─ test
|     |
|     └─ java
|         |
|         └─ resources
|
|
| └─ pom.xml
|
└─ target
```

POM (Project Object Model)

The `pom.xml` file is the heart of every Maven project. It contains:

- Project information (name, version, description, etc.)
- Dependencies
- Plugins
- Build configurations
- Profiles

Sample pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Maven Build Lifecycle

Maven defines a standard build lifecycle:

Phase	Description
validate	Validate project structure
compile	Compile source code
test	Run tests
package	Package code (JAR/WAR)
verify	Integration tests
install	Install into local repo
deploy	Deploy to remote repo

Maven Commands

Command	Description
<code>mvn clean</code>	Cleans the target directory
<code>mvn compile</code>	Compiles the source code
<code>mvn test</code>	Runs the tests
<code>mvn package</code>	Packages the compiled code
<code>mvn install</code>	Installs the package to the local repository
<code>mvn deploy</code>	Deploys the package to a remote repository
<code>mvn dependency:tree</code>	Shows dependency hierarchy

Maven FAQs (Questions & Answers)

Q1: What is Maven?

A: Maven is a build automation tool used mainly for Java projects to manage dependencies and the project build lifecycle.

Q2: What file does Maven use to configure the project?

A: Maven uses `pom.xml` to define project configuration, dependencies, and plugins.

Q3: What are Maven repositories?

A: Maven repositories are locations where Maven stores and retrieves project dependencies.

Q4: What is the Maven build lifecycle?

A: A series of phases (validate, compile, test, package, verify, install, deploy) that define the stages of building a project.

Q5: What does `mvn clean install` do?

A: It cleans the `target` directory, compiles the code, runs tests, and installs the package to the local repository.

Q6: What is a Maven plugin?

A: A plugin extends Maven's functionality and is used to execute tasks such as compiling code, packaging binaries, generating documentation, etc.

Q7: How to add a dependency in Maven?

A: Add the dependency details inside the `<dependencies>` section of `pom.xml`.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>5.3.9</version>
</dependency>
```

Q8 Explain Maven lifecycles.

A: Three lifecycles:

1. **clean** – preps build environment
2. **default** – validate → compile → test → package → install → deploy
3. **site** – generate project documentation

TECHNOLOGY LEARNER